



資料結構

Data Structure

Lab 09

姓名： 簡紘田

學號： 114310115

Lab09-Q1.(題目編號)

Briefly describe the problem. (題目簡述)

Complete the deleteNode and Add comments to the program to clearly describe its actions.

Code

```
#include <iostream>
using namespace std;

// =====
// 樹的節點類別
// =====
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// =====
// 二元搜尋樹類別
// =====
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
    }
}
```

```

    else if (val > node->data) {
        node->right = insert(node->right, val); // 插入右子樹
    }
    // 如果 val == node->data，不插入（避免重複）

    return node; // 回傳更新後的節點
}

// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false;

    if (node->data == target) return true;
    else if (target < node->data) return search(node->left, target); // 繼續
在左子樹找
    else return search(node->right, target); // 繼續在右子樹找
}

// --- 私有的中序遍歷方法（左-根-右） ---
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return;

    inOrderTraversal(node->left); // 先走訪左子樹
    cout << node->data << " "; // 印出目前節點的資料
    inOrderTraversal(node->right); // 再走訪右子樹
}

// --- 私有的刪除方法 ---
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) return nullptr;

    // 找要刪的節點
    if (val < node->data) {
        node->left = deleteNode(node->left, val);
    }
    else if (val > node->data) {
        node->right = deleteNode(node->right, val);
    }
}

```

```

else {
    // 找到了要刪除的節點

    // 沒有子節點的刪除法
    if (node->left == nullptr && node->right == nullptr) {
        delete node;
        return nullptr; // 沒有子節點，所以直接回傳空
    }

    // 只有一個子節點的刪除法
    else if (node->left == nullptr) { // 只有右邊的子節點
        TreeNode* temp = node->right;
        delete node;
        return temp; // 直接讓子替代根
    }
    else if (node->right == nullptr) { // 只有左邊的子節點
        TreeNode* temp = node->left;
        delete node;
        return temp; // 直接讓子替代根
    }

    // 有兩個子節點的回傳法
    else {
        // 從右子樹找最小值
        TreeNode* successor = findMin(node->right);

        // 用替補值替代目前節點
        node->data = successor->data;

        // 刪除右子樹中的替補節點
        node->right = deleteNode(node->right, successor->data);
    }
}

return node;
}

// --- 私有的找最小值方法 ---

```

```

TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) {
        node = node->left;
    }
    return node;
}

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---
    void insert(int val) {
        root = insert(root, val);
    }

    // --- 公開搜尋方法 ---
    bool search(int target) {
        return search(root, target);
    }

    // --- 公開中序遍歷方法 ---
    void displayInOrder() {
        cout << "樹的中序遍歷（排序後）結果：" << endl;
        inOrderTraversal(root);
        cout << endl;
    }

    // --- 公開刪除方法 ---
    void deleteValue(int val) {
        root = deleteNode(root, val);
    }
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料

```

```

int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
int n = sizeof(values) / sizeof(values[0]);

// 把資料插入樹中
for (int i = 0; i < n; ++i) {
    bst.insert(values[i]);
}

// 讓使用者輸入要插入的鍵值
int key;
cout << "請輸入要搜尋/新增的鍵值：";
cin >> key;

if (bst.search(key)) {
    cout << "值 " << key << " 已存在樹中，不新增。" << endl;
}
else {
    cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
    bst.insert(key);
}

// 顯示目前樹的中序遍歷結果
bst.displayInOrder();

// 讓使用者輸入要刪除的鍵值
int deleteKey;
cout << "請輸入要刪除的鍵值：";
cin >> deleteKey;

// 刪除節點
bst.deleteValue(deleteKey);

// 顯示刪除後的樹
cout << "刪除後的樹（中序遍歷）：" << endl;
bst.displayInOrder();

return 0;
}

```

Result

```
#include <iostream>
using namespace std;

// 樹的節點類別
// 節點的類別
class TreeNode {
public:
    int data; // 節點儲存的資料
    TreeNode* left; // 指向左子節點
    TreeNode* right; // 指向右子節點
};

// 節點類別：指定資料並初始化左右指標為空
TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}

// 二元搜尋樹類別
// 二元搜尋樹類別
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // ... 私有的遞迴插入方法 ...
    void insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }
        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
        }
        // 如果 val == node->data，不插入（避免重複）
        return node; // 回傳更新後的節點
    }

public:
    BinarySearchTree() { root = nullptr; }
    void insert(int val) { insert(root, val); }
    void search(int val) { search(root, val); }
    void remove(int val) { remove(root, val); }
    void print() { print(root); }
};

// 搜尋方法
void BinarySearchTree::search(TreeNode* node, int val) {
    if (node == nullptr) {
        cout << "未找到任何節點。" << endl;
        return;
    }
    if (node->data == val) {
        cout << "找到節點: " << val << endl;
    }
    search(node->left, val);
    search(node->right, val);
}

// 刪除方法
void BinarySearchTree::remove(TreeNode* node, int val) {
    if (node == nullptr) {
        return;
    }
    if (node->data == val) {
        if (node->left == nullptr) {
            return node->right;
        }
        if (node->right == nullptr) {
            return node->left;
        }
        int min = node->right->data;
        node->data = min;
        remove(node->right, min);
    }
    else if (val < node->data) {
        remove(node->left, val);
    }
    else if (val > node->data) {
        remove(node->right, val);
    }
    return node;
}

// 印出方法
void BinarySearchTree::print(TreeNode* node) {
    if (node == nullptr) {
        return;
    }
    print(node->left);
    cout << node->data << " ";
    print(node->right);
}

int main() {
    BinarySearchTree tree;
    tree.insert(3);
    tree.insert(4);
    tree.insert(5);
    tree.insert(7);
    tree.insert(8);
    tree.insert(9);
    tree.insert(11);
    tree.insert(12);
    tree.insert(13);
    tree.insert(15);
    tree.search(4);
    tree.remove(4);
    tree.print();
    return 0;
}
```

```
#include <iostream>
using namespace std;

// 樹的節點類別
// 節點的類別
class TreeNode {
public:
    int data; // 節點儲存的資料
    TreeNode* left; // 指向左子節點
    TreeNode* right; // 指向右子節點
};

// 節點類別：指定資料並初始化左右指標為空
TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}

// 二元搜尋樹類別
// 二元搜尋樹類別
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // ... 私有的遞迴插入方法 ...
    void insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }
        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
        }
        // 如果 val == node->data，不插入（避免重複）
        return node; // 回傳更新後的節點
    }

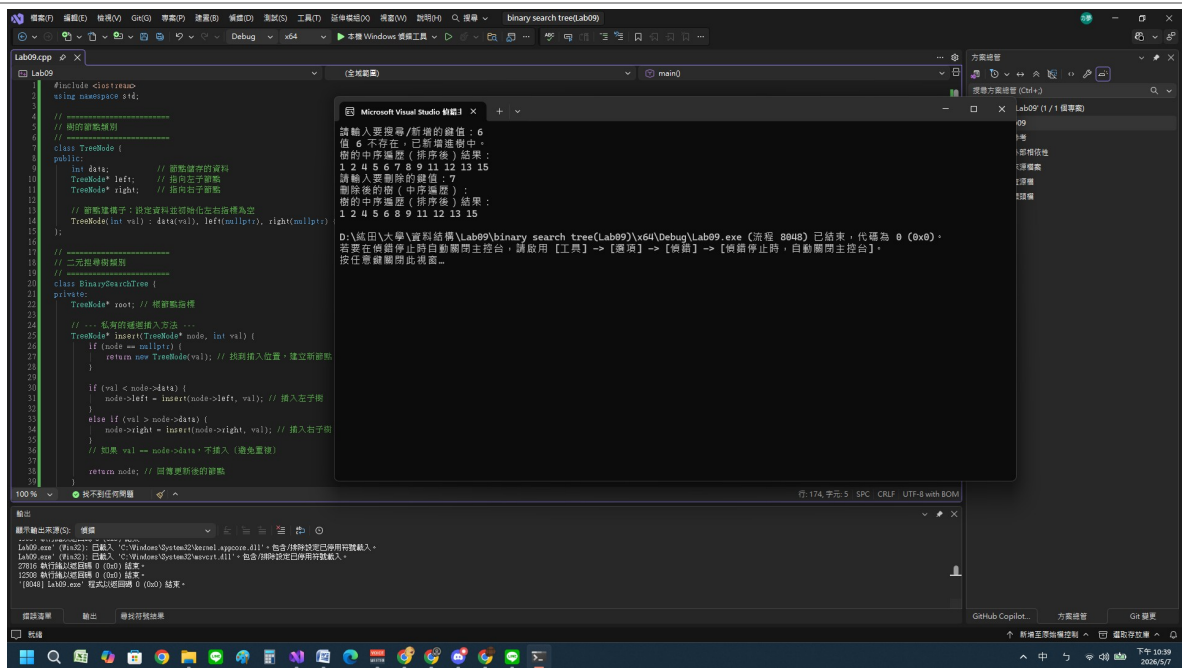
public:
    BinarySearchTree() { root = nullptr; }
    void insert(int val) { insert(root, val); }
    void search(int val) { search(root, val); }
    void remove(int val) { remove(root, val); }
    void print() { print(root); }
};

// 搜尋方法
void BinarySearchTree::search(TreeNode* node, int val) {
    if (node == nullptr) {
        cout << "未找到任何節點。" << endl;
        return;
    }
    if (node->data == val) {
        cout << "找到節點: " << val << endl;
    }
    search(node->left, val);
    search(node->right, val);
}

// 刪除方法
void BinarySearchTree::remove(TreeNode* node, int val) {
    if (node == nullptr) {
        return;
    }
    if (node->data == val) {
        if (node->left == nullptr) {
            return node->right;
        }
        if (node->right == nullptr) {
            return node->left;
        }
        int min = node->right->data;
        node->data = min;
        remove(node->right, min);
    }
    else if (val < node->data) {
        remove(node->left, val);
    }
    else if (val > node->data) {
        remove(node->right, val);
    }
    return node;
}

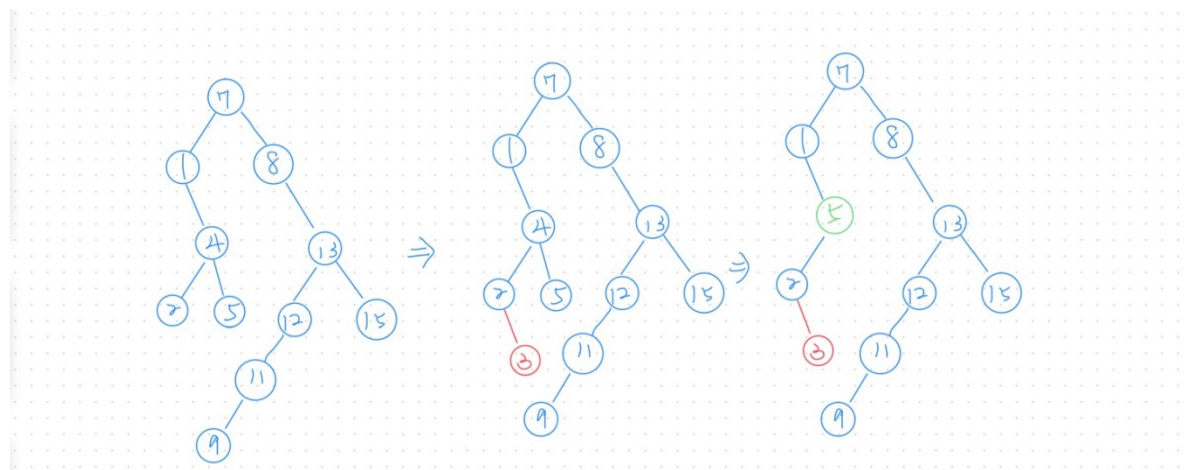
// 印出方法
void BinarySearchTree::print(TreeNode* node) {
    if (node == nullptr) {
        return;
    }
    print(node->left);
    cout << node->data << " ";
    print(node->right);
}

int main() {
    BinarySearchTree tree;
    tree.insert(3);
    tree.insert(4);
    tree.insert(5);
    tree.insert(7);
    tree.insert(8);
    tree.insert(9);
    tree.insert(10);
    tree.insert(11);
    tree.insert(12);
    tree.insert(13);
    tree.insert(15);
    tree.search(10);
    tree.remove(10);
    tree.print();
    return 0;
}
```

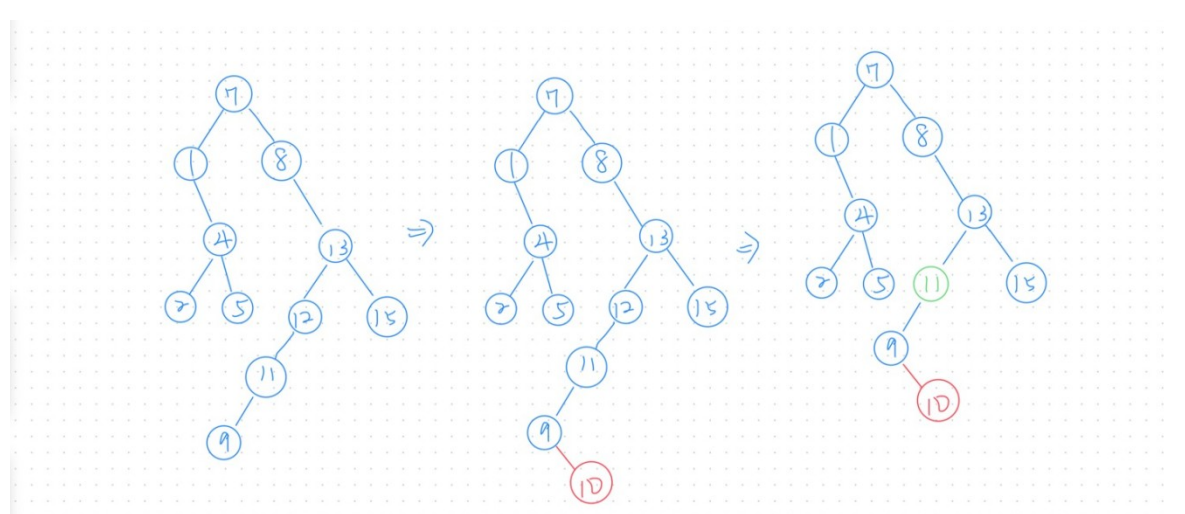


Discussion

第一題：插入 3 刪除 4



第二題：插入 10 刪除 12



第三題：插入 6 刪除 7

